ESPECIFICAÇÃO SDD - TASKANALYZER

Bootcamp III - Entrega Inicial | Especificação Técnica e Governança de Contexto

1. Visão Geral e Contrato de Negócio (SDD)

1.1 Identificação

Estudante	Gabriel Bernardo Dias Alves
Curso	Ciência da Computação
Polo/Turma	Turma B
E-mail institucional	gabriel.bernardo@sempreceub.com
Data de elaboração	13/09/2026
Versão da especificação	1.0

Esta especificação define, de forma executável e verificável, o comportamento esperado do módulo TaskAnalyzer e servirá como fonte de verdade para a geração, validação e homologação do código nas fases seguintes do projeto.

1.2 Propósito do Módulo TaskAnalyzer

O TaskAnalyzer é um módulo de análise de tarefas destinado a gerar métricas objetivas de produtividade a partir de registros de execução. O módulo deverá:

- Analisar tarefas executadas e gerar métricas de produtividade.
- Calcular o tempo médio de conclusão por prioridade e no consolidado geral.
- Identificar a taxa de atraso, considerando tarefas concluídas após o prazo definido.
- Gerar indicadores agrupados por prioridade: baixa, média e alta.
- Fornecer métricas que apoiem a tomada de decisão e a melhoria contínua.

1.3 Contrato Executável de Interface

As entradas, saídas e regras a seguir constituem o contrato que o código deverá seguir estritamente. Qualquer alteração posterior deverá ser versionada, justificada e revisada.

1.3.1 Entradas (Inputs)

Campo	Tipo	Obrigatório	Descrição / Restrições
id_tarefa	int	Sim	Identificador único da tarefa (> 0).
data_criacao	datetime	Sim	Data/hora de criação em UTC.
data_inicio	datetime	Não	Data/hora de início da execução.
data_conclusao	datetime	Não	Data/hora de conclusão da execução.
prazo	datetime	Sim	Data/hora limite para conclusão.
prioridade	str enum	Sim	Valores permitidos: baixa, média, alta.
status	str enum	Sim	Valores permitidos: concluída, pendente, cancelada.

1.3.2 Saídas (Outputs)

Campo	Tipo	Descrição
tempo_medio_conclusao_min	float	Tempo médio de conclusão em minutos, por prioridade e no consolidado geral.
taxa_atraso_percentual	float	Percentual de tarefas concluídas após o prazo, por prioridade e no consolidado geral.
quantidade_tarefas	int	Total de tarefas consideradas na análise, por prioridade e no consolidado geral.
indicadores_por_prioridade	dict	Estrutura com métricas agrupadas por prioridade: baixa, média e alta.

1.3.3 Regras de Negócio e Restrições

- Considerar apenas tarefas com status "concluída" para o cálculo do tempo de conclusão e da taxa de atraso.
- Calcular o tempo de conclusão pela diferença data_conclusao - data_inicio, em minutos.
- Considerar uma tarefa em atraso quando data_conclusao > prazo.
- Ignorar tarefas com datas inconsistentes, por exemplo, conclusão anterior ao início.
- Tratar explicitamente a divisão por zero quando não houver tarefas concluídas.
- Utilizar UTC como fuso horário padrão.
- Arredondar tempos e percentuais para duas casas decimais.

2. Especificação de Cenários de Aceite e Test Harness

2.1 Cenários de Aceite no formato Dado - Quando - Então

Cenário 1 - Sucesso: métricas corretas

Dado: um conjunto de tarefas válidas, com diferentes prioridades, prazos e status, contendo registros concluídos com datas consistentes.

Quando: o analisador de tarefas for executado.

Então: o sistema deverá retornar métricas corretas, incluindo tempo médio de conclusão por prioridade e geral, taxa de atraso correta por prioridade e geral e quantidade de tarefas correta por prioridade.

Cenário 2 - Exceção/Erro: entradas inválidas

Dado: uma entrada com datas inválidas, como conclusão anterior ao início, ou uma situação em que não existam tarefas concluídas e o cálculo possa resultar em divisão por zero.

Quando: o analisador de tarefas for executado.

Então: o sistema deverá tratar a condição de forma controlada, gerando uma exceção específica com mensagem clara e objetiva, sem comportamento indefinido.

2.2 Planejamento do Test Harness

Os cenários de aceite serão transformados, na Fase 2, em testes automatizados com pytest. O objetivo é tornar a especificação verificável e impedir que código fora do contrato seja aceito.

Etapa	Aplicação no projeto
Cenários de aceite	Descrever o comportamento esperado do sistema e servir como referência funcional.
Casos de teste	Decompor cada cenário em casos específicos, com dados de entrada e resultados esperados.

Etapa	Aplicação no projeto
Testes automatizados (pytest)	Implementar testes com pytest, usar fixtures quando necessário e validar saídas, tipos e exceções.
Execução e validação contínua	Executar os testes automaticamente; qualquer falha deverá bloquear o avanço para a etapa seguinte.
Relatórios e evidências	Registrar resultados e garantir rastreabilidade entre cenário, teste e comportamento implementado.

3. Governança de Contexto e Regras para Agentes de IA (CONTEXT_RULES)

As regras desta seção deverão ser fornecidas ao assistente de IA antes da geração ou alteração de código. Elas funcionam como restrições persistentes para manter alinhamento, consistência, segurança e qualidade.

3.1 Diretrizes Arquiteturais Obrigatórias

- Utilizar Python 3.11 ou superior.
- Utilizar type hints em todas as funções e métodos.
- Seguir PEP 8 e boas práticas de código limpo.
- Adotar Google Style docstrings em módulos, classes, funções e parâmetros.
- Manter funções pequenas, coesas e com um único propósito, respeitando o princípio Single Responsibility (SRP).
- Realizar tratamento de exceções específico, com mensagens claras.
- Utilizar logs estruturados por meio do módulo padrão logging.
- Produzir código legível, manutenível e com nomes de variáveis e funções descritivos.

3.2 Proibições Explícitas - Regras Restritivas

- Não utilizar bibliotecas externas não autorizadas.
- Não alterar, remover ou adaptar os cenários de teste fornecidos.
- Não modificar a estrutura de pastas definida nesta especificação.
- Não persistir dados em arquivos ou bancos de dados.
- Não alterar a assinatura, incluindo nome e parâmetros, das funções públicas sem autorização.
- Não gerar código sem testes correspondentes para novas funcionalidades.
- Não inserir código duplicado ou sem necessidade.
- Não assumir comportamentos não especificados no contrato de negócio.

3.3 Regras de Interação com a IA

- Fornecer sempre o contexto completo antes da solicitação.
- Validar se a IA compreendeu o contrato antes de solicitar a geração de código.
- Solicitar explicações quando houver dúvidas na solução proposta.
- Revisar criticamente todo código gerado.
- Rejeitar respostas que violem as proibições explícitas ou o contrato de negócio.
- Manter o arquivo CONTEXT_RULES.md como fonte de verdade para os agentes de IA.

4. Arquitetura do Repositório e Preparação para Git/GitHub

4.1 Árvore do Repositório prevista para a Fase 2

```
sdd-task-analyzer/
├── README.md                  # Visão geral do projeto
├── CONTEXT_RULES.md           # Regras persistentes para a IA
├── specs/
│   └── task_analyzer_spec.md  # Especificação SDD da Fase 1
├── tests/
│   └── test_harness.py        # Testes automatizados com pytest
├── src/
│   └── task_analyzer.py       # Código gerado via IA e homologado
├── .gitignore                 # Arquivos e pastas ignoradas
└── requirements.txt           # Dependências autorizadas
```

4.2 Plano de Homologação Humana

Todo código gerado por IA será submetido a revisão crítica humana antes de ser aceito e versionado. A homologação seguirá os passos abaixo:

1. Executar completamente os testes automatizados para validação funcional.
2. Verificar a conformidade do código com o contrato de negócio e com o arquivo CONTEXT_RULES.md.
3. Avaliar qualidade de código, legibilidade, tratamento de erros e aderência aos padrões definidos.
4. Executar testes manuais complementares em casos de borda quando necessário.
5. Somente após aprovação humana, realizar commit no repositório.

A IA será utilizada como ferramenta de execução e apoio à produtividade; a decisão final de aceite, a homologação e a responsabilidade técnica permanecerão humanas.

4.3 Estratégia de Versionamento para a Fase 2

- Adotar Git Flow simplificado, com branches main, develop e feature/*.
- Realizar commits pequenos, frequentes e descritivos.
- Utilizar Pull Requests para revisão e aprovação.
- Exigir revisão antes do merge.
- Criar tags para releases importantes.

ADENDO — VERSÕES 1.1 E 1.2 DA ESPECIFICAÇÃO

Bootcamp III - TaskAnalyzer | Fases 2 e 3 | Gabriel Bernardo Dias Alves

> As páginas anteriores são o documento SDD original da Fase 1 (versão 1.0, 13/09/2026), exportado do Google Docs sem alterações, como registro histórico. Este adendo resume o que mudou depois. O contrato **vigente** é o arquivo `specs/task_analyzer_spec.md` (versão 1.2), e a versão legível por máquina é `specs/requirements.yaml`.

1. Histórico de versões

Versão	Data	Origem	Resumo
1.0	13/09/2026	Fase 1 (Google Docs)	Especificação inicial, cenários de aceite e CONTEXT_RULES (páginas anteriores).
1.1	17/10/2026	Fase 2	Alinhamento ao enunciado da Fase 2: métricas em horas, totais por status, exceção `TaskValidationError` e retorno 0.0 sem tarefas concluídas.
1.2	25/10/2026	Fase 3	Estrutura final do repositório (CI, Docker, specs) e branch `develop`. Contrato de negócio inalterado.

2. Mudanças da versão 1.0 para a 1.1

Os conflitos abaixo foram percebidos na revisão humana, ao comparar este documento com o enunciado da Fase 2, e decididos pelo autor antes da geração do código, em vez de deixar a IA escolher sozinha.

Ponto	Versão 1.0 (Fase 1)	Versão 1.1 (vigente desde a Fase 2)
Unidade do tempo	`tempo_medio_conclusao_min`, em minutos	`tempo_medio_conclusao_horas`, em horas (RN-03)
Contagens	`quantidade_tarefas`	`total_tarefas`, `total_concluidas`, `total_pendentes` e `total_canceladas`, no geral e por prioridade
Datas inconsistentes	Regra dizia "ignorar"; Cenário 2 dizia "gerar exceção"	Lança `TaskValidationError` (VAL-06), para não esconder erros de dados
Nenhuma tarefa concluída (divisão por zero)	Cenário 2 previa exceção	Retorna 0.0 sem exceção; a divisão por zero é evitada explicitamente (RN-06)
`data_inicio` ausente	Não definido	Usa `data_criacao` (RN-02)
Detalhes	Não definidos	Concluir no prazo não é atraso (RN-04); arredondamento meio para cima (RN-07); datas sem fuso tratadas como UTC

3. Mudanças da versão 1.1 para a 1.2

A seção 4.1 deste documento definiu a árvore do repositório, e as regras proíbem modificá-la. O enunciado da Fase 3 exige arquivos novos. A inclusão foi autorizada pelo autor e versionada (Pull Request #3), antes de qualquer arquivo novo.

- **Infraestrutura:** `.github/workflows/ci.yml`, `Dockerfile`, `docker-compose.yml` e `.dockerignore`. Não são bibliotecas do código de produção: `src/` continua só com a biblioteca padrão.
- **Especificação:** este PDF (`specs/system_design_document.pdf`) e `specs/requirements.yaml`.
- **Branch `develop`:** prevista na seção 4.3 deste documento e não usada na Fase 2; passa a ser a branch de integração.
- **Contrato inalterado:** entradas, saídas, RN-01 a RN-09, VAL-01 a VAL-07 e cenários de aceite iguais aos da 1.1. Testes e código não mudam.

4. Estrutura final do repositório (v1.2)

```
sdd-task-analyzer/
├── .github/
│   └── workflows/
│       └── ci.yml                    # CI: pytest a cada push e pull request
├── Dockerfile                        # Imagem python:3.11-slim que roda o pytest
├── docker-compose.yml                # Serviço que roda a suíte com um comando
├── CONTEXT_RULES.md                  # Regras de governança para a IA (v1.2)
├── specs/
│   ├── task_analyzer_spec.md         # Contrato vigente (v1.2)
│   ├── system_design_document.pdf    # Este documento
│   └── requirements.yaml             # Requisitos em formato de máquina
├── tests/
│   └── test_harness.py               # Test Harness (inalterado desde a Fase 2)
├── src/
│   └── task_analyzer.py              # Código gerado via IA e homologado
├── .gitignore / .dockerignore
├── requirements.txt                  # Dependência autorizada: pytest
└── README.md
```

5. Esteira de entrega

Etapa	Ferramenta	Regra
Integração contínua	GitHub Actions, `ubuntu-latest`, Python 3.11	Roda `pytest tests/test_harness.py -v` em todo push e pull request para `main` e `develop`. Merge só com build verde.
Portabilidade	Docker (`python:3.11-slim`) e Docker Compose	A mesma suíte roda em container isolado com um comando: `docker compose run --rm testes`.
Fluxo Git	`feature/*` → `develop` → `main`	Cada tema chega por Pull Request com checklist de homologação humana.

6. Rastreabilidade (cenário → teste)

Cenário / regra	Testes em `tests/test_harness.py`
Cenário 1 (RN-01 a RN-05, RN-07, RN-09)	`test_cenario1_*`
Cenário 2 (VAL-01 a VAL-07)	`test_cenario2_*`
Casos de borda (RN-06, RN-08)	`test_borda_*`

Total: 43 testes (26 funções, algumas parametrizadas), 100% aprovados localmente, no GitHub Actions e dentro do container Docker.